Dynamic Programming

Dynamic Programming

- Powerful algorithmic design technique
  - Large class of seemingly exponential problems have a polynomial solution (“only”) via DP
- Particularly for optimization problems (min / max)
- DP ≈ “controlled brute force”

History: Richard E. Bellman (1920-1984)

- Richard Bellman, Eye of the Hurricane: An Autobiography (1984, excerpts from page 159)
  - Where did the name, dynamic programming, come from? The 1950s were not good years for mathematical research.
  - We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word "research"… He would turn red, and he would get violent if people used the term research in his presence. You can imagine how he felt, then, about the term mathematical. Where did the name, dynamic programming, come from? The 1950s were not good years for mathematical research.
  - I felt I had to do something to shield Wilson … from the fact that I was really doing mathematics… What title, what name, could I choose? In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word "programming." I wanted to get across the idea that this was “dynamic,” this was multistage, this was time-varying. I thought, let's kill two birds with one stone.
  - I thought dynamic programming was a good name. It was something not even a Congressman could object to.

Dynamic Programming: Fibonacci Numbers

- Consider the problem of finding n-th Fibonacci Numbers:
- Brute force Recursive Solution
  - F(n) = F(n-1) + F(n-2)
  - F(0) = 0
  - F(1) = 1
- Runtime Analysis
  - T(n) = T(n-1) + T(n-2) + O(1)
  - Exponential time (double exponential in term of input size)
  - T(n) ≥ 2T(n-2)+ O(1) → n/2 levels in the recursion tree → $\boldsymbol{\Omega(2^{n/2})}$
  - $T(n) \leq 2T(n-1) + O(1)$ → n levels in the recursion tree → $\boldsymbol{O(2^n)}$

Recursive algorithm

```
def Fib(n):
    if n == 0 :
        return 0
    if n == 1 :
        return 1
    return Fib(n-1) + Fib(n-2)
```

Dynamic Programming: Memoization

- An efficient implementation of fibonacci: **memoization**
  - It is the same recursive algorithm
- Runtime: O(n)
  - It records the computation that is already done and therefore many of the function calls are not executed, and therefore many of the subproblems are not called again
  - Runtime: #subproblems X (time to solve one subproblem)

```
def Fib(n):
    if F[n] != None:
        return F[n]
    else:
        f = Fib(n-1) + Fib(n-2)

    F[n] = f
    return f
```

Fib(k) only recurses the first time called

for each k

memoized calls → O(1) constant time

we have n non-memoized calls k = n, . . . , 1

Fib(1), . . . Fib(n)

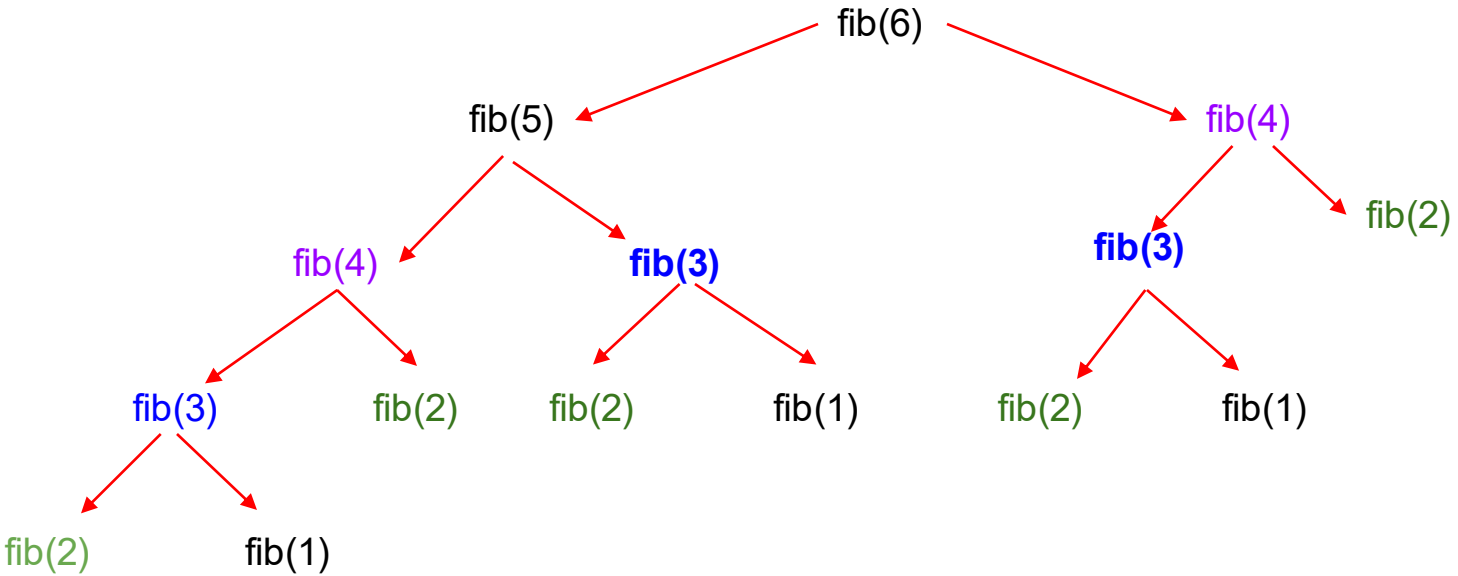
cost of each non-memoized call is constant

(once you ignore recursion)

Dynamic Programming: Fibonacci Numbers

Why the recursive fibonacci is so slow?

Overlapping subproblems



Dynamic Programming: Bottom-up DP Algorithm

- Another efficient implementation of fibonacci
- Runtime: O(n)
  - N-1 integer addition
  - Recursion tree is collapsed

Iterative algorithm

```
def betterFib(n):
    F[0] = 0
    for i in range(2, n):
        F[i] = F[i-1] + F[i-2]

    return F[n]
```

Main idea of dynamic programming:

- solve “subproblems” from smaller to larger (**bottom up**) storing solutions.

Memoization vs. Bottom-Up

Which approach do you think is easier to implement for a given problem?

- Memoization: Top-down approach. Solves the problem recursively and stores results of solved subproblems.
- Bottom-Up: Fills a table iteratively to solve the problem.

Dynamic Programming

- Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems and solving each subproblem just once, storing the solutions for future reference (**memoization** or **bottom-up**).
- **What kinds of problems do you think would benefit from this approach?**
  - (Optimal) Recursive SubStructure
  - Overlapping subproblems
  - Polynomial number of distinct subproblems

Optimal Substructure

- A problem exhibits optimal substructure if an optimal solution to the problem contains optimal solutions to its subproblems.
- The solution to a problem can be expressed in terms of solutions to smaller sub-problems.

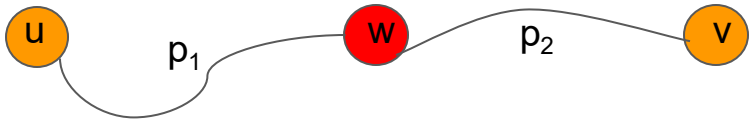
    - Example:
        - Big problems break up into sub-problems.
            - Fibonacci: F(i) for $i \leq n$
            - Fibonacci: $F(i) = F(i-1) + F(i-2)$

Optimal Substructure

- Optimal substructure does not always apply
  - **Shortest path problem**: Find a simple path from u to v consisting of the fewest edge.
    - Has an optimal substructure
  - **longest path**: find a simple path from u to v consisting of the most edges.
    - Simple path means a path with no cycle
    - Does not have an optimal substructure

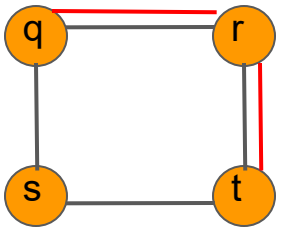
Optimal Substructure: shortest path problem

- **p**: the shortest path between u and v: optimal solution
- Number of edges in p = Number of edges in p_1 + Number of edges in p_2
- **Claim**: p_1 is a shortest path from u to w:
  - If there were another path, say p'_1 from u to w with fewer edges then we could cut out p_1 and paste in p'_1 to produce a path $p' = p'_1 + p_2$ with fewer edges → contradiction: p_1 is an optimal solution or shortest path
  - Similarly we can show p_2 is the shortest path from w to v
- Therefore we can find a shortest path from u to v by considering all intermediate vertices w, finding a shortest path from u to w, finding a shortest path from w to v, choosing the w with overall shortest path

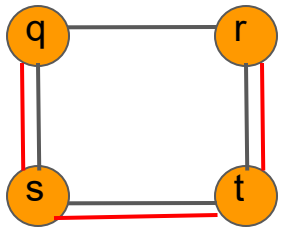


Optimal Substructure: longest path problem

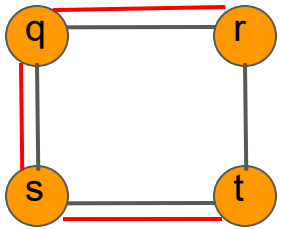
- Longest path from **q** to **t**: q-r-t
- Is q-r a longest path from q to r? No, longest path is q-s-t-r
- Is r-t a longest path from r to t? No, longest path is r-q-s-t
- This problem does not have optimal substrcuture→ we cannot use dynamic programming /greedy here



Longest path from q to t



Longest path from q to r



Longest path from r to t

Overlapping Subproblems

- The problem can be broken down into subproblems that are reused several times.
  - The sub-problems overlap
  - Fibonacci:
    - Both F[i+1] and F[i+2] directly use F[i].
  - And lots of different F[i+x] indirectly use F[i].

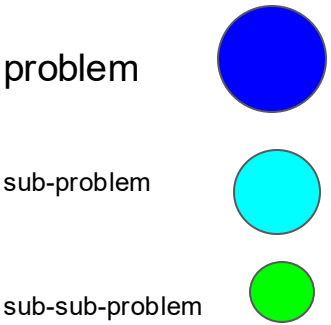
Implications: Save time (have more efficient algorithm) by solving a sub-problem just once and recording the result in a table from which a solution to the original problem can be obtained

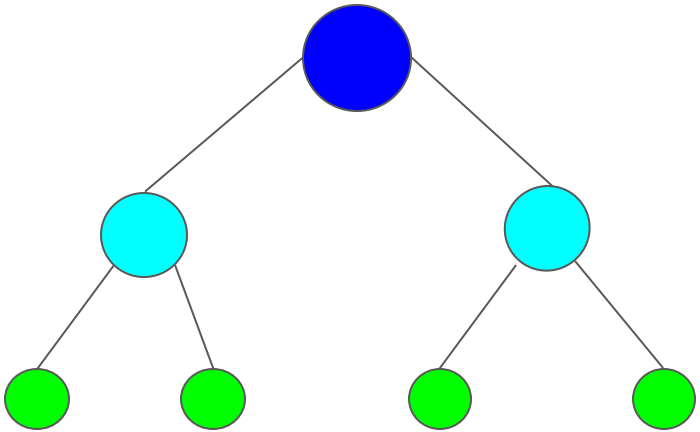
Question

Why is brute force inefficient for problems with overlapping subproblems?

Divide-and-conquer

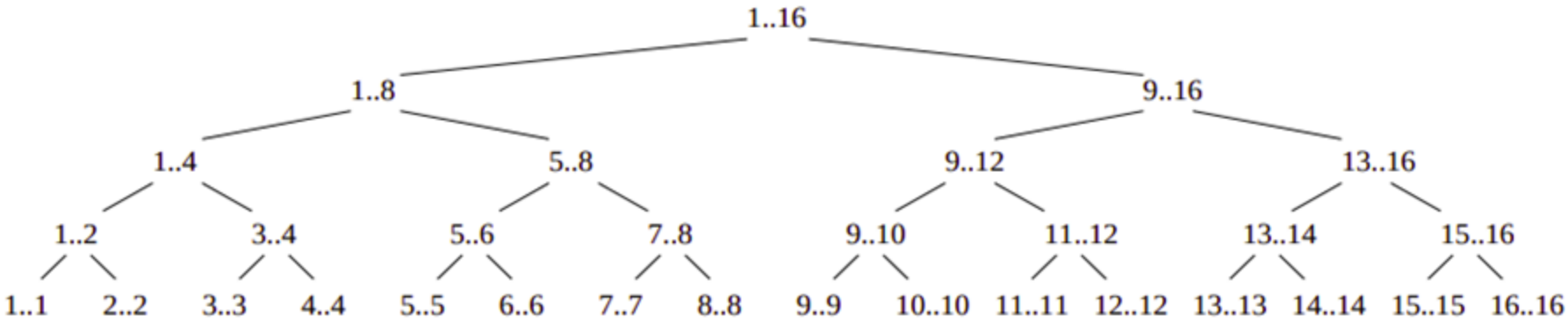
Break up a problem into **independent** subproblems; solve each subproblem; combine solutions to subproblems to form solution to original problem.





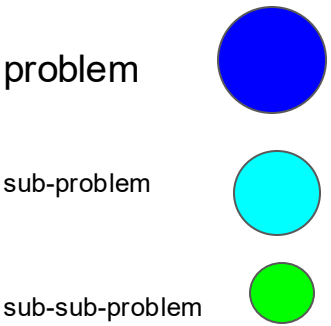
Subproblem graph view: Mergesort

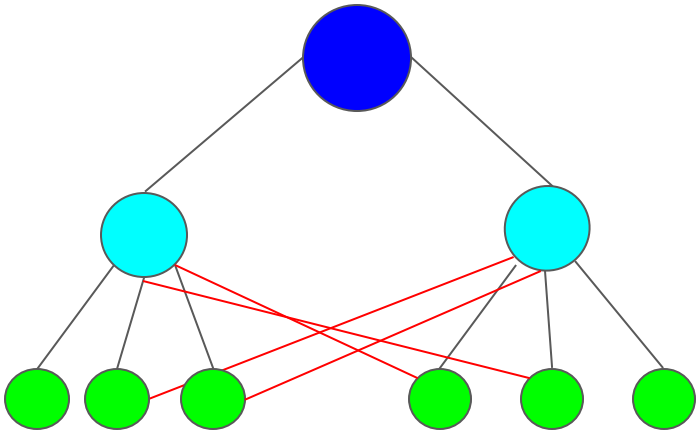
- No repeated subproblems
  - Dynamic Programming not good for “Divide & Conquer” algorithms



Dynamic programming

Break up a problem into a series of independent but **overlapping** subproblems; combine solutions to smaller subproblems to form solution to large subproblem.





Polynomial Subproblems

- The number of **distinct** subproblems is small enough to be evaluated in polynomial time.